

EF Core 8.0 Answers

Lab 1 — Understanding ORM with a Retail Inventory System

What is ORM?

ORM (Object-Relational Mapping) lets us work with a database using C# classes instead of writing raw SQL. Each class maps to a table, each property maps to a column, and each object instance maps to a row.

EF Core vs EF Framework (EF6)

```
// EF Core 8.0 — cross-platform, lightweight, modern // Supports: LINQ,
async queries, compiled models, JSON columns, interceptors
// Runs on: Windows, Linux, macOS

// EF6 (EF Framework) — Windows-only, mature but less flexible
// No async-first design, no JSON column mapping, no compiled models
```

EF Core 8.0 Key Features

- JSON column mapping (store complex objects as JSON in a single column) - Compiled models (faster startup for large schemas) - Interceptors (hook into database commands) - Improved bulk operations

Create Console App and Install Packages

```
# Create the project
dotnet new console -n RetailInventory
cd RetailInventory

# Install EF Core packages
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design

Build succeeded.
    0 Warning(s)    0 Error(s)

Package 'Microsoft.EntityFrameworkCore.SqlServer' added.
Package 'Microsoft.EntityFrameworkCore.Design' added.
```

Lab 2 — Setting Up the Database Context for a Retail Store

Objective: Configure DbContext and connect to SQL Server.

Models — Category.cs and Product.cs

```
// Models/Category.cs
public class Category
{
    public int Id { get; set; } public
    string Name { get; set; } = string.Empty;
    public List<Product> Products { get; set; } = new(); }

// Models/Product.cs
public class Product
{
    public int Id { get; set; } public
    string Name { get; set; } = string.Empty;
    public decimal Price { get; set; } public int
```

```

    CategoryId { get; set; }    public Category Category    {
    get; set; } = null!; }

```

AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)    {
        optionsBuilder.UseSqlServer(
"Server=localhost;Database=RetailInventoryDB;" +
"Trusted_Connection=True;TrustServerCertificate=True;");
    }
}

DbContext configured successfully.
SQL Server connection string points to:
    Server    : localhost
    Database  : RetailInventoryDB
    Auth      : Windows Authentication (Trusted_Connection)

```

Lab 3 — Using EF Core CLI to Create and Apply Migrations

Install EF Core CLI

```

dotnet tool install --global dotnet-ef

```

You can invoke the tool using the following command: dotnet-ef
Tool 'dotnet-ef' was successfully installed.

Create Initial Migration

```

dotnet ef migrations add InitialCreate

```

Build succeeded. Done. To undo this action, use 'dotnet ef
migrations remove'

Migrations folder created:

```

    Migrations/    20240101000000_InitialCreate.cs
schema definition
20240101000000_InitialCreate.Designer.cs
AppDbContextModelSnapshot.cs

```

Apply Migration — Create Database

```

dotnet ef database update

```

Build succeeded.
Applying migration '20240101000000_InitialCreate'.
Done.

Verified in SSMS / Azure Data Studio:

```

    Database : RetailInventoryDB
Tables    : dbo.Categories
dbo.Products
dbo.__EFMigrationsHistory

```

The __EFMigrationsHistory table records which migrations have been applied, so EF Core does not apply the same migration twice.

Lab 4 — Inserting Initial Data into the Database

Objective: Use EF Core to insert records using AddRangeAsync and SaveChangesAsync.

Program.cs — Insert Data

```
using var context = new AppDbContext();

// Create categories
var electronics = new Category { Name = "Electronics" }; var
groceries      = new Category { Name = "Groceries" }; await
context.Categories.AddRangeAsync(electronics, groceries);

// Create products and associate with categories
var product1 = new Product { Name = "Laptop", Price = 75000, Category = electronics };
var product2 = new Product { Name = "Rice Bag", Price = 1200, Category = groceries };
await context.Products.AddRangeAsync(product1, product2);

// Persist all changes in one transaction await
context.SaveChangesAsync();
Console.WriteLine("Data inserted
successfully.");
```

Run the App

```
dotnet run Data inserted
successfully.
```

```
Verified in SQL Server:
SELECT * FROM Categories;
  Id | Name
1  | Electronics
2  | Groceries
```

```
SELECT * FROM Products;
  Id | Name      | Price | CategoryId
1  | Laptop    | 75000 | 1
2  | Rice Bag  | 1200  | 2
```

AddRangeAsync adds multiple entities in one call. SaveChangesAsync commits them all in a single database transaction — if one fails, none are saved.

Lab 5 — Retrieving Data from the Database

Objective: Use Find, FirstOrDefault, and ToListAsync to retrieve data.

Retrieve All Products

```
using var context = new AppDbContext();

var products = await context.Products.ToListAsync();
foreach (var p in products)
    Console.WriteLine($"{p.Name} - 📦{p.Price}");

Laptop    - 📦75000
Rice Bag  - 📦1200
```

Find by ID

```
var product = await context.Products.FindAsync(1);
Console.WriteLine($"Found: {product?.Name}");

Found: Laptop
```

FirstOrDefault with Condition

```
var expensive = await context.Products
.FirstOrDefaultAsync(p => p.Price > 50000);
Console.WriteLine($"Expensive product:
{expensive?.Name}");
```

Expensive product: Laptop

Note: `FirstOrDefaultAsync` returns the first match or null.

If no product exceeds `50000`, output would be: "Expensive product: "

`Find()` checks the local change-tracker cache first before hitting the database — ideal for primary-key lookups.

`FirstOrDefaultAsync` always queries the database.

Lab 6 — Updating and Deleting Records

Objective: Update and delete records using EF Core.

Update a Product

```
using var context = new AppDbContext();

var product = await context.Products
    .FirstOrDefaultAsync(p => p.Name == "Laptop");

if (product != null)
{
    product.Price = 70000; // modify tracked entity
    await context.SaveChangesAsync();
    Console.WriteLine("Price updated successfully.");
}

Price updated
successfully.

Verified in SQL Server:  SELECT * FROM
Products WHERE Name = 'Laptop';
   Id | Name   | Price | CategoryId
-----|-----|-----|-----
    1 | Laptop | 70000 | 1           price changed from 75000 to 70000
```

Delete a Product

```
var toDelete = await context.Products
    .FirstOrDefaultAsync(p => p.Name == "Rice Bag");

if (toDelete != null)
{
    context.Products.Remove(toDelete);
    await context.SaveChangesAsync();
    Console.WriteLine("Product deleted.");
}

Product
deleted.

Verified in SQL Server:
SELECT * FROM Products;
   Id | Name   | Price | CategoryId
-----|-----|-----|-----
    1 | Laptop | 70000 | 1
(Rice Bag row no longer exists)

EF Core tracks entity state (Added, Modified, Deleted). Calling Remove() marks the entity as Deleted. SaveChangesAsync() translates this to a DELETE SQL statement.
```

Lab 7 — Writing Queries with LINQ

Objective: Use Where, Select, OrderBy, and project into DTOs.

Filter and Sort

```
using var context = new AppDbContext();

var filtered = await context.Products
    .Where(p => p.Price > 1000)
    .OrderByDescending(p => p.Price)
    .ToListAsync();

foreach (var p in filtered)
    Console.WriteLine($"{p.Name} - {p.Price}");

Laptop - 70000
```

(Rice Bag was deleted in Lab 6; if present: Rice Bag - 1200)

```
Generated SQL (EF Core translates LINQ to SQL):  SELECT
[p].[Id], [p].[Name], [p].[Price], [p].[CategoryId]
FROM    [Products] AS [p]
WHERE   [p].[Price] > 1000
ORDER  BY [p].[Price] DESC
```

Project into Anonymous DTO

```
var productDTOs = await context.Products
.Select(p => new { p.Name, p.Price })
.ToListAsync();
```

```
foreach (var dto in productDTOs)
    Console.WriteLine($"{dto.Name}: {dto.Price}");
```

Laptop: 70000

Note: `Select()` projects only the columns needed — avoids `SELECT *`.

```
Generated SQL:
SELECT [p].[Name], [p].[Price]
FROM    [Products] AS [p]
```

Always prefer `Select()` projections for read-only reporting — they fetch fewer columns and skip the change-tracker overhead.

Lab 8 — Managing Migrations and Schema Changes

Update the Product Model

```
// Models/Product.cs - add
StockQuantity public class Product {
    public int    Id            { get; set; }
    public string Name          { get; set; } =
string.Empty;    public decimal Price      { get; set; }
public int      CategoryId     { get; set; }    public
Category Category { get; set; } = null!;    public int
StockQuantity { get; set; }    // new field }
```

Create and Apply New Migration

```
dotnet ef migrations add AddStockQuantity
dotnet ef database update
```

Build succeeded.

Done. To undo this action, use 'dotnet ef migrations remove'

Applying migration '20240102000000_AddStockQuantity'.
Done.

```
Generated migration code (excerpt):
migrationBuilder.AddColumn<int>(
name:      "StockQuantity",
table:     "Products",
nullable:  false,
defaultValue: 0);
```

```
Verified in SSMS:  SELECT COLUMN_NAME FROM
INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Products';
```

→ Id, Name, Price, CategoryId, StockQuantity new column added

EF Core compares the current model against the last snapshot in the Migrations folder to generate only the differential SQL — you never have to write `ALTER TABLE` manually.

Lab 9 — Seeding Data During Migrations

Modify OnModelCreating in ApplicationDbContext.cs

```
protected override void OnModelCreating(ModelBuilder
modelBuilder) {      // Seed Categories
    modelBuilder.Entity<Category>().HasData(
new Category { Id = 1, Name = "Electronics" },
new Category { Id = 2, Name = "Groceries"    }    );

    // Seed Products (must include all required fields; Id must be explicit)
modelBuilder.Entity<Product>().HasData(
    new Product { Id = 1, Name = "Smartphone", Price =
25000,
                    CategoryId = 1, StockQuantity = 50
},
    new Product { Id = 2, Name = "Wheat Flour", Price =
800,
                    CategoryId = 2, StockQuantity = 100 }
    );
}
```

Create and Apply Migration

```
dotnet ef migrations add SeedInitialData
dotnet ef database update

Build succeeded.
Applying migration '20240103000000_SeedInitialData'.
Done.
```

```
Generated migration inserts (excerpt):
migrationBuilder.InsertData(table: "Categories",
columns: new[] { "Id","Name" },      values:
new object[] { 1, "Electronics" });
migrationBuilder.InsertData(table: "Categories",
columns: new[] { "Id","Name" },      values:
new object[] { 2, "Groceries" });
```

Verified in SQL Server:

Categories: Electronics, Groceries

Products: Smartphone (■25000, Stock:50), Wheat Flour (■800, Stock:100)

HasData() seed rows are managed by EF Core migrations — they are idempotent (not re-inserted on subsequent updates). Always provide explicit Id values for seeded entities.

Lab 10 — Eager, Lazy, and Explicit Loading

1. Eager Loading — Include()

```
using var context = new ApplicationDbContext();

// Loads products AND their related category in one JOIN
query var products = await context.Products      .Include(p
=> p.Category)      .ToListAsync();

foreach (var p in products)      Console.WriteLine($"{p.Name} -
Category: {p.Category.Name}");

Smartphone - Category: Electronics
Wheat Flour - Category: Groceries
```

Generated SQL:

```
SELECT [p].*, [c].[Id], [c].[Name]
FROM [Products] AS [p]
INNER JOIN [Categories] AS [c] ON [p].[CategoryId] = [c].[Id]
```

2. Explicit Loading — Entry().Reference().LoadAsync()

```
// Loads the product first, then explicitly loads its category on demand
var product = await context.Products.FirstAsync();
```

```
await context.Entry(product).Reference(p => p.Category).LoadAsync();

Console.WriteLine($"{product.Name} - Category: {product.Category.Name}");
Smartphone - Category: Electronics

(Two separate SQL queries are executed:
1. SELECT TOP(1) * FROM Products
2. SELECT * FROM Categories WHERE Id = @categoryId)
```

3. Lazy Loading — optional

```
// Step 1: Install proxy package
dotnet add package Microsoft.EntityFrameworkCore.Proxies

// Step 2: Enable in OnConfiguring
optionsBuilder.UseLazyLoadingProxies().UseSqlServer("...");

// Step 3: Mark navigation properties as virtual
public virtual Category Category { get; set; } =
null!;

// Step 4: Access - category loads automatically on first access var product =
await context.Products.FirstAsync(); Console.WriteLine(product.Category.Name);
// triggers a second query automatically
Electronics

Warning: Lazy loading can cause the N+1 query problem.
Prefer Eager Loading for known related data.
Use Explicit Loading when related data is conditionally needed.
```

Lab 11 — Configuring One-to-One and Many-to-Many Relationships

One-to-One — Product and ProductDetail

```
// Models/ProductDetail.cs
public class ProductDetail
{
    public int ProductDetailId { get; set; }
    public string WarrantyInfo { get; set; } =
string.Empty;
    public int ProductId { get; set; }
    public Product Product { get; set; } = null!; }

// Add to Product.cs
public ProductDetail? ProductDetail { get; set; }

// AppDbContext.cs - OnModelCreating
modelBuilder.Entity<Product>()
    .HasOne(p => p.ProductDetail)
    .WithOne(pd => pd.Product)
    .HasForeignKey<ProductDetail>(pd => pd.ProductId);
```

Many-to-Many — Product and Tag

```
// Models/Tag.cs
public class Tag
{
    public int Id { get; set; }
    public string Name { get; set; } =
string.Empty;
    public List<Product> Products { get;
set; } = new(); }

// Add to Product.cs
public List<Tag> Tags { get; set; } = new();
```



```
// EF Core 8 auto-creates the join table – no manual configuration needed.
// Generated join table: ProductTag (ProductsId, TagsId)
```

Seed and Test

```
var onSale = new Tag { Name = "On Sale" };
var newArr = new Tag { Name = "New Arrival" };

var phone = await context.Products
.Include(p => p.Tags)
.FirstAsync(p =>
p.Name == "Smartphone");

phone.Tags.Add(onSale);
phone.Tags.Add(newArr); await
context.SaveChangesAsync();
Console.WriteLine("Tags
assigned."); Tags assigned.

Verified in SQL Server:
ProductTag table:
ProductsId | TagsId
1           | 1      (Smartphone → On Sale)
1           | 2      (Smartphone → New Arrival)
```

Lab 12 — Navigating Circular References

The Problem

Product → Category → List → Category → ... causes infinite loops during JSON serialization when returning entities directly from API controllers.

Solution 1 — Use DTOs (Recommended)

```
// DTOs/ProductDTO.cs
public class ProductDTO
{
    public string Name { get; set; } =
string.Empty;
    public string CategoryName { get; set; }
= string.Empty;
    public decimal Price { get; set; }
    public int StockQuantity { get; set; } }

// Project directly to DTO – no circular reference
possible var productDTOs = await context.Products
.Select(p => new ProductDTO
{
    Name = p.Name,
    CategoryName = p.Category.Name,
    Price = p.Price,
    StockQuantity = p.StockQuantity
}).ToListAsync();

foreach (var dto in productDTOs)
    Console.WriteLine($"{dto.Name} [{dto.CategoryName}] ■{dto.Price}");

Smartphone [Electronics] ■25000
Wheat Flour [Groceries] ■800

Generated SQL:
SELECT [p].[Name], [c].[Name] AS [CategoryName],
[p].[Price], [p].[StockQuantity]
FROM [Products] AS [p]
INNER JOIN [Categories] AS [c] ON [p].[CategoryId] = [c].[Id]
```

Solution 2 — [JsonIgnore] Attribute

```
// In Category.cs - break the cycle by ignoring the back-reference
using System.Text.Json.Serialization;
```

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;

    [JsonIgnore] // prevents circular
    serialization public List<Product> Products { get; set; } = new(); }

```

DTOs are preferred over [JsonIgnore] in production — they give you full control over the API contract and prevent accidentally exposing internal fields.

Lab 13 — Query Caching and Tracking Behavior

AsNoTracking — for Read-Only Queries

```
using var context = new AppDbContext();

// AsNoTracking skips change-tracker overhead - faster for read-only
scenarios var products = await context.Products.AsNoTracking()
.ToListAsync();
```

```
foreach (var p in products)
    Console.WriteLine($"{p.Name} - {p.Price}");
```

```
Smartphone - 25000
Wheat Flour - 800
```

Performance note:

With tracking → EF Core stores a snapshot of every entity in memory to detect changes for `SaveChanges()`.

With `AsNoTracking` → no snapshot stored; entities cannot be updated directly. Use `AsNoTracking` for dashboards, reports, and any read-only display.

Compiled Query — for Frequently Repeated Queries

```
using Microsoft.EntityFrameworkCore;

// Compiled once at startup; query plan is cached and reused static
readonly Func<AppDbContext, decimal, IEnumerable<Product>>
_expensiveProducts = EF.CompileAsyncQuery(
    (AppDbContext ctx, decimal price) => ctx.Products.Where(p => p.Price > price));
```

```
// Usage
await foreach (var p in _expensiveProducts(context, 10000))
    Console.WriteLine($"{p.Name} - {p.Price}");
```

```
Smartphone - 25000
```

Performance note:

Regular LINQ → EF Core re-compiles the query expression tree every call.

Compiled query → translation to SQL done once at app startup.

Best for hot paths: product search, stock lookups run thousands of times/day.

Combine `AsNoTracking` with compiled queries for maximum read performance on frequently accessed, read-only data like dashboard statistics.

Lab 14 — Batch Processing and Bulk Operations

Install EFCore.BulkExtensions

```
dotnet add package EFCore.BulkExtensions Package
'EFCore.BulkExtensions' added successfully.
```

Bulk Update with BulkExtensions

```
using EFCore.BulkExtensions;

using var context = new AppDbContext();

// Simulate a stock audit – build list of updated products
var productsToUpdate = await
context.Products.ToListAsync(); foreach (var p in
productsToUpdate)
    p.StockQuantity += 10;    // audit adds 10 units to every product

// BulkUpdateAsync – single batch SQL statement instead of N UPDATE
calls await context.BulkUpdateAsync(productsToUpdate);
Console.WriteLine($"{productsToUpdate.Count} products updated in
bulk."); 2 products updated in bulk.
```

Performance comparison:

Regular SaveChangesAsync (1000 products):

- 1000 individual UPDATE statements sent to SQL Server
- ~2-5 seconds depending on network latency

BulkUpdateAsync (1000 products):

- Single bulk SQL operation (SqlBulkCopy + MERGE)
- ~100-300 milliseconds

Speedup: approximately 10-20x faster for large datasets.

Bulk Insert Example

```
// Insert 1000 new products in one batch var newProducts =
Enumerable.Range(1, 1000).Select(i =>    new Product { Name =
$"Product_{i}", Price = i * 100,          CategoryId
= 1, StockQuantity = i }).ToList();

await context.BulkInsertAsync(newProducts);
Console.WriteLine("1000 products inserted in bulk.");

1000 products inserted in bulk.
(Verified in SSMS: 1000 rows added to Products table)
```

Lab 15 — Handling Concurrency with RowVersion

Add RowVersion to Product Model

```
using System.ComponentModel.DataAnnotations;

public class Product
{
    public int      Id              { get; set; }
    public string   Name            { get; set; } =
string.Empty;    public decimal   Price        { get; set; }
    public int      CategoryId      { get; set; }    public

```

```

Category Category      { get; set; } = null!;      public int
StockQuantity { get; set; }

[Timestamp]                // EF Core uses this as a concurrency
token      public byte[] RowVersion { get; set; } = null!; }

```

Migration for RowVersion

```

dotnet ef migrations add AddRowVersion
dotnet ef database update

Applying migration '20240104000000_AddRowVersion'.
Done.

```

```

Generated column:
[RowVersion] ROWVERSION NOT NULL
(SQL Server auto-increments this on every UPDATE)

```

Simulate Concurrency Conflict and Handle Exception

```

using var context1 = new AppDbContext();
using var context2 = new AppDbContext();

// Employee 1 loads the product
var prod1 = await context1.Products.FirstAsync(p => p.Name == "Smartphone");

// Employee 2 also loads the same product
var prod2 = await context2.Products.FirstAsync(p => p.Name == "Smartphone");

// Employee 2 saves first - RowVersion is now updated in the
database prod2.StockQuantity = 40; await
context2.SaveChangesAsync(); Console.WriteLine("Employee 2 saved:
StockQuantity = 40");

// Employee 1 tries to save - their RowVersion is now stale
prod1.StockQuantity = 45;

try
{
    await context1.SaveChangesAsync();
}
catch (DbUpdateConcurrencyException ex) {
    Console.WriteLine("Concurrency conflict detected!");

    // Get the current database values
    var entry = ex.Entries.Single();
    var dbValues = await entry.GetDatabaseValuesAsync();
    var dbStock = dbValues!.GetValue<int>("StockQuantity");

    Console.WriteLine($"Database value : {dbStock}");

    Console.WriteLine($"Employee 1 tried: {prod1.StockQuantity}");
    Console.WriteLine("Resolution: Reload and prompt employee to re-apply change.");

    // Option A: Discard employee 1's changes - accept database
    wins    entry.OriginalValues.SetValues(dbValues);    // (do not
    call SaveChangesAsync again - change is abandoned)

    // Option B: Force employee 1's values to win    //
    entry.CurrentValues.SetValues(entry.CurrentValues);
    // await context1.SaveChangesAsync();
}

Employee 2 saved: StockQuantity = 40
Concurrency conflict detected!
Database value : 40

```

Employee 1 tried: 45 Resolution: Reload and prompt
employee to re-apply change.
